



# CSE 2201 : Algorithms

## Lecture 04: Order of growth

Atanu Shuvam Roy



Class Code  
GVWTHYJV



Lecturer,  
Dept. of CSE, ULAB, Dhaka

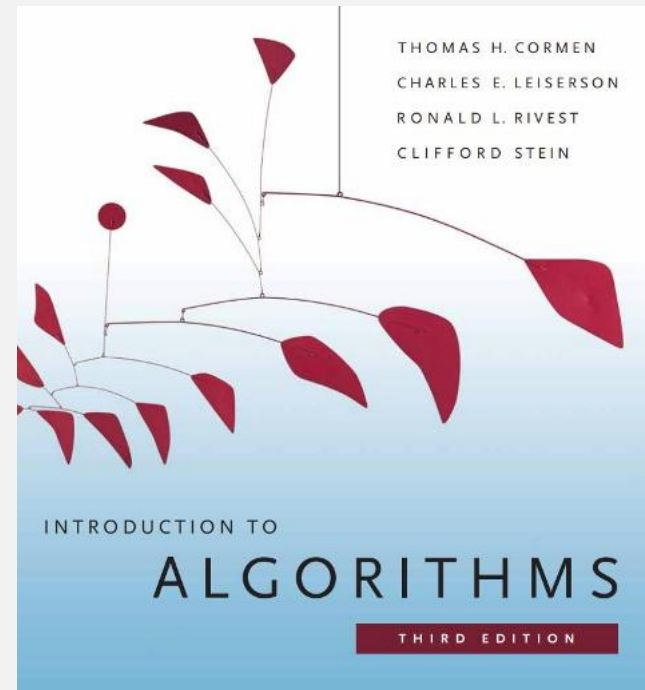
EMAIL:  
**[atanu.shuvam@ulab.edu.bd](mailto:atanu.shuvam@ulab.edu.bd)**

---

Fall 2025

# Reference Book

- Introduction to Algorithms, 3rd edition. T.H. Cormen, C.E. Leiserson, R.L. Rivest, C. Stein



# Distribution of Marks

| Assessments                    | %   |
|--------------------------------|-----|
| Mid Term Examination           | 25  |
| Final Examination              | 40  |
| Attendance & class performance | 10  |
| Assignment & Quizzes           | 25  |
| Total                          | 100 |





PRINCETON  
UNIVERSITY



# Binary search: mathematical analysis

---

**Proposition.** Binary search uses at most  $1 + \lg N$  key compares to search in a sorted array of size  $N$ .

**Def.**  $T(N) \equiv$  # key compares to binary search a sorted subarray of size  $\leq N$ .

**Binary search recurrence.**  $T(N) \leq T(N/2) + 1$  for  $N > 1$ , with  $T(1) = 1$ .

↑  
left or right half

↑  
possible to implement with one  
2-way compare (instead of 3-way)

**Pf sketch.**

$$T(N) \leq T(N/2) + 1$$

given

$$\leq T(N/4) + 1 + 1$$

apply recurrence to first term

$$\leq T(N/8) + 1 + 1 + 1$$

apply recurrence to first term

...

$$\leq T(N/N) + 1 + 1 + \dots + 1$$

stop applying,  $T(1) = 1$

$$= 1 + \lg N$$

# Recurrence Relation

- **1. What is a Recurrence Relation?**
- A **recurrence relation** expresses a function in terms of **smaller inputs** of the same function.
- In simpler words:
- It defines a problem's total cost based on the cost of smaller subproblems.
- Formally, it looks like:
- $T(n) = a \cdot T\left(\frac{n}{b}\right) + f(n)$
- Where:
- $T(n)$  :time to solve a problem of size  $n$
- $a$ : number of subproblems created
- $n/b$ : size of each subproblem
- $f(n)$  :extra work done outside of recursion (like splitting, merging, etc.)

# Recurrence Relation

- **2. Why we need it (Running time connection)**
- When an algorithm **calls itself recursively**, we can't just directly count the total time — because each call does some work and then calls itself again.
- So we write an equation describing **how total time builds up**.
- Example:  
Merge Sort
- $T(n) = 2T\left(\frac{n}{2}\right) + O(n)$
- Explanation:
- Two recursive calls on half the array  $\rightarrow 2T(n/2)$
- Plus  $O(n)$  time to merge the results.

# Recurrence Relation

- **3. How to Solve a Recurrence**

- We want a **closed form** (non-recursive) expression for  $T(n)$ .

- There are several methods:

- **(a) Substitution / Expansion**

- Keep expanding until the base case.

- Example:

- $T(n) = 2T(n/2) + n$

- Expand:

- $T(n) = 2[2T(n/4) + n/2] + n = 4T(n/4) + 2n$

- $= 8T(n/8) + 3n$

- After  $k$  expansions:

- $T(n) = 2^k T(n/2^k) + kn$

- When  $n/2^k = 1 \Rightarrow k = \log_2 n$ :

- $T(n) = nT(1) + n \log_2 n = O(n \log n)$

-  So Merge Sort runs in  **$O(n \log n)$**  time.



## (b) Master Theorem

If your recurrence is in the form:

$$T(n) = aT\left(\frac{n}{b}\right) + f(n)$$

you can apply the **Master Theorem** directly:

$$T(n) = aT\left(\frac{n}{b}\right) + f(n)$$

| Case | Condition                                                              | Result                               |
|------|------------------------------------------------------------------------|--------------------------------------|
| 1    | $f(n) = O(n^{\log_b a - \epsilon})$                                    | $T(n) = \Theta(n^{\log_b a})$        |
| 2    | $f(n) = \Theta(n^{\log_b a})$                                          | $T(n) = \Theta(n^{\log_b a} \log n)$ |
| 3    | $f(n) = \Omega(n^{\log_b a + \epsilon})$ , and<br>regularity condition | $T(n) = \Theta(f(n))$                |

# Let's save master theorem for later

Because I don't want you to die yet

# Binary search: mathematical analysis

---

**Proposition.** Binary search uses at most  $1 + \lg N$  key compares to search in a sorted array of size  $N$ .

**Def.**  $T(N) \equiv$  # key compares to binary search a sorted subarray of size  $\leq N$ .

**Binary search recurrence.**  $T(N) \leq T(N/2) + 1$  for  $N > 1$ , with  $T(1) = 1$ .

↑  
left or right half

↑  
possible to implement with one  
2-way compare (instead of 3-way)

**Pf sketch.**

$$T(N) \leq T(N/2) + 1$$

given

$$\leq T(N/4) + 1 + 1$$

apply recurrence to first term

$$\leq T(N/8) + 1 + 1 + 1$$

apply recurrence to first term

...

$$\leq T(N/N) + 1 + 1 + \dots + 1$$

stop applying,  $T(1) = 1$

$$= 1 + \lg N$$

# Binary Search Recurrence Relation solve (Using Expansion/Substitution)

- **Step 1: First expansion**

- Start by expanding  $T(n/2)$  from the recurrence itself:
- $T(n) = [T(n/2)] + 1$
- Replace  $T(n/2)$  using the same formula:
- $T(n/2) = T(n/4) + 1$
- Substitute back:
- $T(n) = [T(n/4) + 1] + 1 = T(n/4) + 2$



# Binary Search Recurrence Relation solve (Using Expansion/Substitution)

- **Step 2: Expand again**
  - Expand one more time:
  - $T(n/4) = T(n/8) + 1$
  - So:
  - $T(n) = T(n/8) + 3$
- **Step 3: Continue the pattern**
  - After expanding  $k$  times, the pattern becomes clear:
  - $T(n) = T\left(\frac{n}{2^k}\right) + k$
- **Step 4: Stop condition (base case)**
  - The recursion stops when the problem size becomes **1**:
  - $\frac{n}{2^k} = 1$
  - Solve for  $k$ :
  - $2^k = n \Rightarrow k = \log_2 n$

# Binary Search Recurrence Relation solve (Using Expansion/Substitution)

- **Step 5: Substitute  $k = \log_2 n$** 
  - Plug into the expanded formula:
  - $T(n) = T(1) + \log_2 n$
  - We know  $T(1) = 1$ , so:
  - $T(n) = 1 + \log_2 n$
- **Step 6: Simplify asymptotically**
  - Asymptotically (for large  $n$ ):
  - $T(n) = O(\log n)$

# Why $1 + \lg n$ ?

- **1. How binary search works**

- Binary search repeatedly splits the array in half:
- Check the middle element.
- If it's the target, stop.
- Otherwise, pick the left or right half and repeat.
- At each step, the size of the search interval **roughly halves**.

- **2. Maximum number of halvings**

- Let's say the array has  $n$  elements.
- After  $k$  steps, the remaining search interval has at most:
- $\frac{n}{2^k}$  elements.
- Binary search stops when the interval has **1 element**, because then we either find the target or conclude it's not there.
- So we solve:
- $\frac{n}{2^k} \leq 1 \Rightarrow 2^k \geq n \Rightarrow k \geq \log_2 n$
- Since  $k$  must be an integer, we take the **ceiling**:
- $k = \lceil \log_2 n \rceil$

- **3. Why “1 + ...” instead of just  $\log_2 n$ ?**

- The  $\log_2 n$  counts **the number of times you halve the array**.
- The **first comparison** (checking the middle element in the very first step) is **not counted in the halving**.
- So the total comparisons = **first check + number of halvings**:
- $1 + \lceil \log_2 n \rceil$

# Another Proof of the same thing

- Let's expand it:

$$T(N) \leq T(N/2) + 1$$

$$\leq [T(N/4) + 1] + 1 = T(N/4) + 2$$

- $\leq [T(N/8) + 1] + 2 = T(N/8) + 3$

$\vdots$

$$\leq T(N/2^k) + k$$

- We stop when  $N/2^k = 1 \implies k = \log_2 N$ .
- At that point,  $T(1) = 1$  (one comparison for the single element), so:
- $T(N) \leq 1 + \log_2 N$

# An $N^2 \log N$ algorithm for 3-SUM

## Sorting-based algorithm.

- Step 1: **Sort** the  $N$  (distinct) numbers.
- Step 2: For each pair of numbers  $a[i]$  and  $a[j]$ , **binary search** for  $-(a[i] + a[j])$ .

input

30 -40 -20 -10 40 0 10 5

sort

-40 -20 -10 0 5 10 30 40

binary search

(-40, -20) 60

(-40, -10) 50

(-40, 0) 40

(-40, 5) 35

(-40, 10) 30

⋮ ⋮

(-40, 40) 0

⋮ ⋮

(-20, -10) 30

⋮ ⋮

(-10, 0) 10

⋮ ⋮

( 10, 30) -40

( 10, 40) -50

( 30, 40) -70

only count if  
 $a[i] < a[j] < a[k]$   
to avoid  
double counting

**Analysis.** Order of growth is  $N^2 \log N$ .

- Step 1:  $N^2$  with insertion sort.
- Step 2:  $N^2 \log N$  with binary search.



# Comparing programs

---

**Hypothesis.** The sorting-based  $N^2 \log N$  algorithm for 3-SUM is significantly faster in practice than the brute-force  $N^3$  algorithm.

| N     | time (seconds) |
|-------|----------------|
| 1,000 | 0.1            |
| 2,000 | 0.8            |
| 4,000 | 6.4            |
| 8,000 | 51.1           |

ThreeSum.java

| N      | time (seconds) |
|--------|----------------|
| 1,000  | 0.14           |
| 2,000  | 0.18           |
| 4,000  | 0.34           |
| 8,000  | 0.96           |
| 16,000 | 3.67           |
| 32,000 | 14.88          |
| 64,000 | 59.16          |

ThreeSumDeluxe.java

**Guiding principle.** Typically, better order of growth  $\Rightarrow$  faster in practice.

# Let's Work a Little

- <https://www.geeksforgeeks.org/dsa/find-a-triplet-that-sum-to-a-given-value/>

# Reference

- <http://coursera.org/learn/algorithms-part1/>



# Thank You

[atanu.Shuvam@ulab.edu.bd](mailto:atanu.Shuvam@ulab.edu.bd)

University of Liberal Arts Bangladesh  
**Department of Computer Science & Engineering**